

Monacos Indoor Health Hub - Comprehensive Feature Documentation

1. Project Overview

Monacos Indoor Health Hub is a full-stack, state-of-the-art indoor environmental monitoring system. It aggregates real-time sensor data, calculates advanced health and air quality metrics, predicts future environmental conditions, and presents all data through a modern, responsive React dashboard. A key innovation is the **Health Guardian**, an AI-powered assistant capable of explaining sensor data, forecasting trends, and answering user queries using both voice and text.

2. Backend Architecture & Features

The backend is built using **Python and FastAPI**, providing robust, high-performance API endpoints. It utilizes **SQLite** for long-term data persistence and in-memory caching for ultra-fast, real-time reads.

A. Data Ingestion & Storage (`main.py`, `db.py`)

- **API Endpoint:** `POST /api/ingest` (and `/data` for Arduino backwards compatibility)
- **Logic:**
 - Accepts a structured JSON payload containing real-time device ID, temperature, humidity, PM2.5, PM10, noise, light, altitude, pressure, CO2, and VOCs.
 - Generates a server-side timestamp.
 - **In-Memory Cache:** Instantly updates `DEVICE_STATE` (for live readings) and `DEVICE_HISTORY` (circular buffer) for ultra-low latency dashboard retrieval.
 - **Long-term Persistence:** Every reading is logged to `sensor_readings` table in `monacos.db` (SQLite) for historical analytics and AI model training.
- **Functions:** `ingest()`, `ensure_device_exists()`

B. Device Management & Status Tracking

- **API Endpoint:** `GET /api/devices`
- **Logic:**
 - Validates active devices against the database.
 - Determines an "online" or "offline" status based on an `ONLINE_TIMEOUT` (30 seconds) constant. If a device has not sent data within 30 seconds, it's flagged as disconnected.
- **Functions:** `list_devices()`

C. Authentication System (`auth.py`)

- **API Endpoints:** `POST /auth/signup`, `POST /auth/login`, `GET /auth/me`
- **Logic:**
 - Implements secure, token-based authentication (JWT - Json Web Tokens).
 - Uses `bcrypt` for secure, salted password hashing before storing in the database.
 - Returns short-lived access tokens to clients to securely access protected routes.
- **Functions:** `get_password_hash()`, `verify_password()`, `create_access_token()`, `decode_access_token()`

D. AI Health Guardian & Forecasting Engine (`agent_engine.py`)

- **API Endpoint:** `POST /api/chat`
- **Logic:**
 - **Predictive Modeling:** Incorporates a localized Multiple Linear Regression model from `scikit-learn`. Evaluates the last 100 environmental data points, extracting `time_ordinal` and `hour` to compute continuous trends and cyclical daily patterns. Generates a 5-hour scientific forecast.
 - **Dynamic Context Injection:** Merges live sensor readings and historical data into a structured system prompt, allowing the AI to "see" exactly what is happening in the room.
 - **LLM Routing Architecture:**
 1. Primarily routes requests to Google Gemini (`gemini-2.0-flash` with fallbacks) for highly responsive, intelligent responses.
 2. Falls back to OpenAI GPT-4o-mini if configured.
 3. Triggers a smart **Offline/Regex rule-based fallback** engine if API quotas are exceeded, ensuring the system can still answer basic questions ("What is the temperature?").
- **Functions:** `predict_with_explanation()`, `get_live_context()`, `run_agent()`

E. Advanced Prediction Engine (`prediction_engine.py`)

- **API Endpoint:** `GET /api/predict/{device_id}`
- **Logic:**
 - Employs a **Recursive Multi-Target Random Forest Regressor**.
 - **Feature Engineering:** Resamples data into 1-hour intervals to eliminate noise. Generates time-based features (`seconds_since_start`, `day_of_week`, `hour`, `is_weekend`) and autoregressive lag features for all metrics.
 - **Forecasting:** Predicts multiple targets (Temp, Humidity, PM2.5, PM10) 24 hours into the future simultaneously.
 - **Compliance Validation:** Automatically cross-references forecasted PM2.5 and PM10 averages against **WHO (World Health Organization)** 24-hour guidelines, and Temp/Humidity against **ASHRAE** comfort standards.
- **Functions:** `fetch_training_data()`, `prepare_features()`, `check_standards()`, `generate_predictions()`

F. Health Score Engine (`health_engine.py`)

- **API Endpoint:** `GET /api/health-score/{device_id}`
- **Logic:** Calculates an absolute 0-100 environmental health score using a weighted deduction algorithm across three categories:
 - **Category A (Respiratory, Max 50 points):** Deducts points heavily for high PM2.5 (>35), PM10, CO2 (>1200), and VOCs.

- **Category B (Thermal, Max 30 points):** Deducts for extreme or uncomfortable temperature and mold-risk level humidity (>70%).
- **Category C (Stressors, Max 20 points):** Deducts for hazardous noise (>75 dB) and poor lighting.
- Maps final score to categories: Good (80+), Moderate (60+), Poor (40+), Hazardous (<40).

- **Functions:** `calculate_health_score()`

G. Air Quality Index (AQI) Calculator (`aqi_engine.py`)

- **API Endpoint:** `GET /api/aqi/{device_id}`
- **Logic:** Utilizes the official US EPA piecewise linear interpolation algorithm to map raw PM2.5 and PM10 mass concentrations to the 0-500 AQI scale. Takes the highest (worst) sub-index to represent the overall AQI.
- **Functions:** `calculate_sub_index()`, `calculate_pm_aqi()`

H. Smart Alerts & Hardware Diagnostics Engine (`alerts_engine.py`)

- **API Endpoint:** `GET /api/alerts/{device_id}`
- **Logic:**
 - Actively scans real-time streams for critical threshold violations (e.g., Temperature > 30°C, Noise > 90 dB).
 - **Interference Deduplication/Diagnostics:** Employs heuristic physical rules to identify false positives. For example, if Temperature > 45°C but VOC/Smoke is low, it flags a "Hardware Interference" alert (likely placed on a laptop exhaust) rather than a fire alert.
 - **Spam Prevention:** Uses an in-memory `ALERT_CACHE` to enforce a 60-second cooldown per alert-type per device.
- **Functions:** `_can_emit()`, `generate_alerts()`

I. Recommendation Engine (`recommendation_engine.py`)

- **API Endpoint:** `GET /api/recommendations/{device_id}`
- **Logic:** Pure rule-based engine mapping current environmental states to actionable, human-readable advice (e.g., "Run air purifier on high" if PM2.5 > 35, or "Use a humidifier" if humidity < 30%).
- **Functions:** `generate_recommendations()`

3. Frontend Architecture & Features

The client-side application is built using **React**, **TypeScript**, **Vite**, **Tailwind CSS**, and **Framer Motion**. It consumes the FastAPI endpoints to render dynamic, animated, real-time views.

A. Central State Management & Real-time Integration (`SensorContext.tsx`)

- **Logic:**
 - `SensorProvider` acts as the single source of truth for the entire application.
 - Initiates concurrent asynchronous API fetch calls (`Promise.all`) to gather the latest sensor readings, history, alerts, AQI, and health score every 5 seconds (Polling).
 - Maintains `latest`, `historicalData`, `alerts`, `recommendations`, and `aqi` in state, distributing them to consumer UI components efficiently without prop-drilling.
 - Exposes `injectSensorData` for manual or Bluetooth-based overrides.

B. Interactive Dashboard (`Dashboard.tsx`)

- **Logic:** The central hub where all high-level components are arranged using a responsive CSS Grid layout. Utilizes `framer-motion` for stagger-delay entrance animations. Connects the `HealthScoreCard`, `AQICard`, `SensorGrid`, and dynamic sidebars.

C. Voice-Enabled AI Chat Interface (`DashboardChat.tsx`)

- **Logic:**
 - Subscribes to the AI backend endpoint.
 - Provides a floating, highly-animated glass-morphism chat window.
 - **Speech-to-Text:** Integrates the native browser `webkitSpeechRecognition` API. Allows users to simply click the microphone icon, dictate their question, and auto-submit the form.
 - **Text-to-Speech:** Integrates `window.speechSynthesis`. Automatically speaks the AI's response aloud using available system voices, creating a hands-free interactive experience.
 - **Dynamic UI:** Renders animated loading states and distinct user vs. assistant chat bubbles.

D. Environmental Visualization Components (`components/`)

- **HealthScoreCard:** Renders a massive, visually striking radial gauge depicting the 0-100 score, heavily leveraging the `health_engine` output.
- **AQICard:** Displays the EPA standard AQI prominently alongside its health category (Good, Moderate, Unhealthy).
- **SensorGrid & SensorCard:** A mapped array of individual metric cards (Temperature, Humidity, PM2.5, Noise, Light) displaying exact numeric values dynamically sourced from `SensorContext`.
- **Alerts & Recommendations Panels:** Sidebar lists rendering real-time incoming push notifications and actionable advice, color-coded by severity and priority.
- **TrendChart:** A historical time-series graph utilizing dynamic client-side charting libraries to visualize the previous 7 days of environmental variations.

E. Analytics & Settings Features (`Analytics.tsx`, `Settings.tsx`)

- **Analytics View:** Dedicated page for deep-dive historical graphing, cross metric overlay (e.g. Temperature vs Humidity correlations), and exporting tools (fetching CSV from `/api/export/{id}`).
- **Settings View:** Houses user preferences, profile updates, threshold configuration interfaces, and global application options handled by standard state hooks and form submissions.

F. API Services Layer (`services/`)

- **Logic:** Modularized proxy functions (`api.ts`, `aqiService.ts`) enforcing separation of concerns. All React components call these abstracted asynchronous functions rather than utilizing raw `fetch` calls directly within `.tsx` files. Makes HTTP protocol upgrades (e.g. WebSockets) trivial to implement centrally.